```
import tensorflow as tf
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
dataset=pd.read_csv("Churn_Modelling.csv")
x=dataset.iloc[:,3:13]
y=dataset.iloc[:,13]
x.head()
```

	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	Esti
0	619	France	Female	42	2	0.00	1	1	1	
1	608	Spain	Female	41	1	83807.86	1	0	1	
2	502	France	Female	42	8	159660.80	3	1	0	
3	699	France	Female	39	1	0.00	2	0	0	
4	850	Spain	Female	43	2	125510.82	1	1	1	

Next steps: Generate code with x New interactive sheet

```
geog=pd.get_dummies(x['Geography'],drop_first=True)
gen=pd.get_dummies(x['Gender'],drop_first=True)
x=x.drop(['Geography','Gender'],axis=1)
x=pd.concat([x,geog,gen],axis=1)
x.head()
```

	CreditScore	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	German
0	619	42	2	0.00	1	1	1	101348.88	Fals
1	608	41	1	83807.86	1	0	1	112542.58	Fals
2	502	42	8	159660.80	3	1	0	113931.57	Fals
3	699	39	1	0.00	2	0	0	93826.63	Fals
4	850	43	2	125510.82	1	1	1	79084.10	Fals

Next steps: Generate code with x New interactive sheet

```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=0)
from sklearn.preprocessing import StandardScaler
sc=StandardScaler()
x_train=sc.fit_transform(x_train)
x_test=sc.transform(x_test)
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import LeakyReLU,PReLU,ELU, ReLU
from tensorflow.keras.layers import Dropout
classifier=Sequential()
classifier.add(Dense(units=11,activation='relu'))
classifier.add(Dense(units=7,activation='relu'))
classifier.add(Dense(units=6,activation='relu'))
classifier.add(Dense(units=1,activation='sigmoid'))
```

```
classifier.compile(optimizer='adam',loss='binary_crossentropy',metrics=['accuracy'])
model_history=classifier.fit(x_train,y_train,validation_split=0.33,batch_size=10,epochs=100)
```

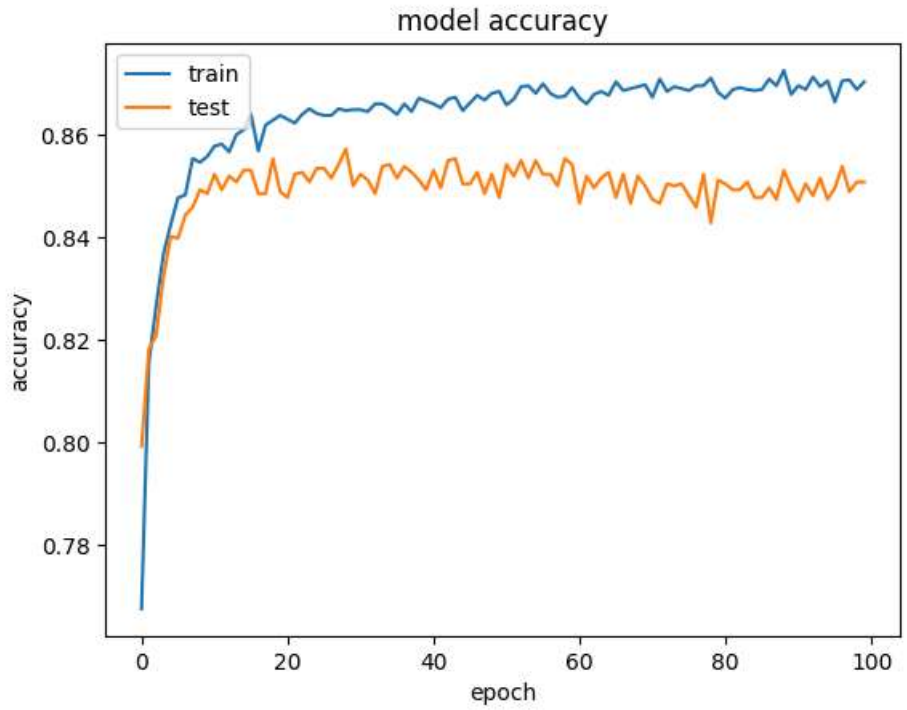
```
536/536 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.8690 - loss: 0.3147 - val_accuracy: 0.8466 - val
Epoch 73/100
536/536 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8674 - loss: 0.3095 - val_accuracy: 0.8504 - val
Epoch 74/100
536/536 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8683 - loss: 0.3126 - val_accuracy: 0.8501 - val
Epoch 75/100
536/536 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8684 - loss: 0.3060 - val_accuracy: 0.8504 - val
Epoch 76/100
536/536 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8668 - loss: 0.3134 - val_accuracy: 0.8482 - val
Epoch 77/100
536/536 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8735 - loss: 0.3024 - val_accuracy: 0.8459 - val
Epoch 78/100
536/536 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8752 - loss: 0.3002 - val_accuracy: 0.8523 - val
Epoch 79/100
536/536 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8757 - loss: 0.3043 - val_accuracy: 0.8429 - val
Epoch 80/100
536/536 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.8624 - loss: 0.3228 - val_accuracy: 0.8512 - val
Epoch 81/100
536/536 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.8742 - loss: 0.2975 - val_accuracy: 0.8504 - val
Epoch 82/100
536/536 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8668 - loss: 0.3141 - val_accuracy: 0.8493 - val
Epoch 83/100
536/536 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8700 - loss: 0.3074 - val_accuracy: 0.8493 - val
Epoch 84/100
536/536 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8685 - loss: 0.3061 - val_accuracy: 0.8508 - val
Epoch 85/100
536/536 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8676 - loss: 0.3112 - val_accuracy: 0.8478 - val
Epoch 86/100
536/536 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8667 - loss: 0.3116 - val_accuracy: 0.8478 - val
Epoch 87/100
536/536 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8794 - loss: 0.3018 - val_accuracy: 0.8497 - val
Epoch 88/100
536/536 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.8742 - loss: 0.3057 - val_accuracy: 0.8474 - val
Epoch 89/100
536/536 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.8835 - loss: 0.2973 - val_accuracy: 0.8531 - val
Epoch 90/100
536/536 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8737 - loss: 0.3045 - val_accuracy: 0.8497 - val
Epoch 91/100
536/536 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8645 - loss: 0.3181 - val_accuracy: 0.8470 - val
Epoch 92/100
536/536 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8777 - loss: 0.2955 - val_accuracy: 0.8504 - val
Epoch 93/100
536/536 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8645 - loss: 0.3200 - val_accuracy: 0.8482 - val
Epoch 94/100
536/536 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8677 - loss: 0.3058 - val_accuracy: 0.8516 - val
Epoch 95/100
536/536 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.8716 - loss: 0.3091 - val_accuracy: 0.8474 - val
Epoch 96/100
536/536 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.8689 - loss: 0.3043 - val_accuracy: 0.8497 - val
Epoch 97/100
536/536 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8752 - loss: 0.3032 - val_accuracy: 0.8538 - val
Epoch 98/100
536/536 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8754 - loss: 0.2961 - val_accuracy: 0.8489 - val
Epoch 99/100
536/536 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8723 - loss: 0.3004 - val_accuracy: 0.8508 - val
Epoch 100/100
536/536 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8712 - loss: 0.3064 - val_accuracy: 0.8508 - val
```

```
model_history.history.keys()
```

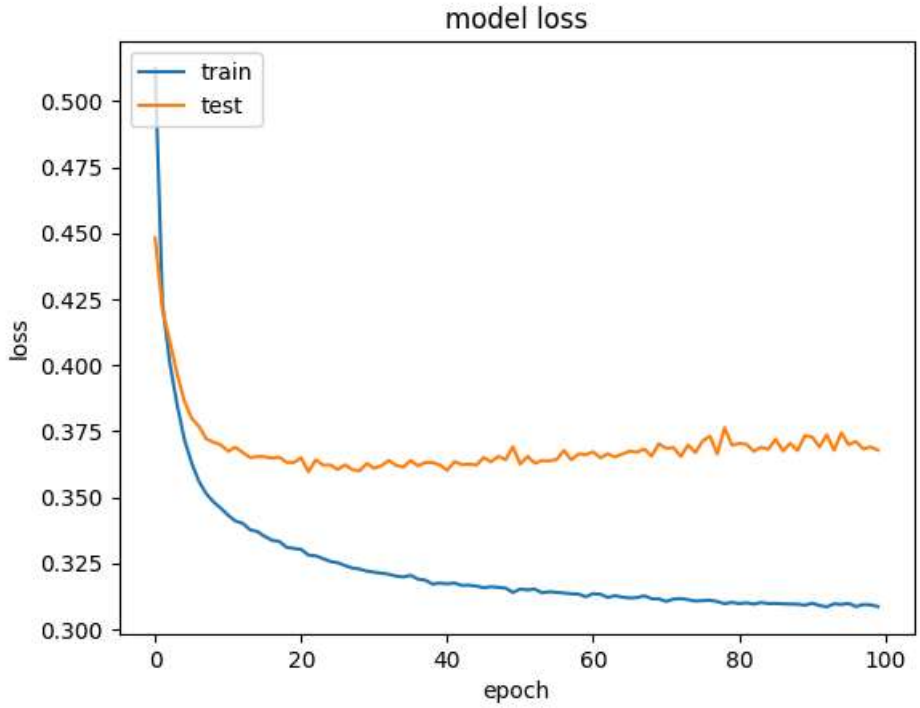
```
dict_keys(['accuracy', 'loss', 'val_accuracy', 'val_loss'])
```

```
plt.plot(model_history.history['accuracy'])
plt.plot(model_history.history['val_accuracy'])
plt.title('model accuracy')
```

```
plt.ylabel('accuracy')
plt.xlabel('epoch')
plt.legend(['train','test'],loc='upper left')
plt.show()
```




```
plt.plot(model_history.history['loss'])
plt.plot(model_history.history['val_loss'])
plt.title('model loss')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.legend(['train','test'],loc='upper left')
plt.show()
```



```
y_pred=classifier.predict(x_test)
y_pred=(y_pred>0.5)
from sklearn.metrics import confusion_matrix
cm=confusion_matrix(y_test,y_pred)
cm
```

```
63/63 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step
array([[1511,   84],
       [ 208,  197]])
```

```
from sklearn.metrics import accuracy_score
score=accuracy_score(y_pred,y_test)
score
```

```
0.854
```

```
classifier.get_weights()
```

```
[array([[ 0.2186396 ,  0.1201136 , -0.15960148,  0.3994254 , -0.21135628,
         -0.22828847, -0.37946028,  0.06017539,  0.2080299 ,  0.45415238,
          0.1272331 ],
        [-1.0346766 ,  0.39185968,  0.19227685,  0.41414446,  0.1331345 ,
         -0.38792896,  0.45471275, -1.0216467 ,  0.7899556 , -0.85624164,
          0.12116347],
        [-0.08877972,  0.36193067,  0.2369397 , -0.65090144,  0.11369176,
          0.25958002, -0.04439744,  0.09628243, -0.05304875, -0.07598241,
          0.20148385],
        [-0.03207764, -0.62003595,  0.18646793, -0.16419439, -0.8459698 ,
          0.4871057 ,  0.18114632, -0.37147263, -0.1999217 , -0.18762206,
         -0.14527948],
        [-0.03872133, -0.11254857,  0.5360236 , -0.0756678 , -0.9696425 ,
          0.7907561 , -0.7262469 ,  0.3024677 ,  0.91269124,  0.02873722,
         -1.1160526 ],
        [ 0.21979454, -0.03001947,  0.11184324, -0.5292659 , -0.02760337,
         -0.09842329, -0.01484863,  0.03967458,  0.03823758, -0.06651454,
          0.1458857 ],
        [ 0.89373255,  0.2549974 , -0.96459854, -0.34030476, -0.3061792 ,
          0.5905916 ,  0.08672089, -0.8942777 , -0.04412013,  0.3045997 ,
         -0.06983693],
        [ 0.09271166, -0.35114202,  0.03781682, -0.39617422,  0.1621186 ,
          0.28603053,  0.38101578, -0.07225528, -0.23354521, -0.66678953,
         -0.05161643],
        [ 0.18267724, -0.18393838, -0.2271889 , -0.04249318,  0.78537846,
          0.34555662,  0.6419331 , -0.01969214,  0.08917508, -0.18100312,
          0.05820687],
        [ 0.06286087,  0.33822122, -0.17882459, -0.14543664, -0.11049394,
          0.31466   , -0.15363213, -0.32187447,  0.15098517, -0.17830274,
          0.25789458],
        [ 0.11373351,  0.44287938,  0.02286373,  0.5344655 , -0.00581094,
          0.24150573,  0.08126663,  0.19608228, -0.01466944,  0.5167219 ,
         -0.02558874]], dtype=float32),
 array([ 0.42082322,  0.5970058 ,  0.04379141, -0.03271658, -0.17377348,
         0.61504775,  0.14124888,  0.87826604,  0.0817462 , -0.01423171,
        -0.05913641], dtype=float32),
 array([[ 0.700018  , -0.5109286 , -0.3267621 , -0.7071778 , -0.15533513,
         -0.257699  , -0.1889044 ],
        [ 0.1769156 ,  0.22208217, -0.3188586 , -0.33495846,  0.47940865,
          0.31949255, -0.6865773 ],
        [ 0.32516524,  0.49899822,  0.37751135,  0.28046373,  0.30722204,
         -0.87822145,  0.67221797],
        [ 0.08777568,  0.60094184,  0.08668336, -0.04842638,  0.28489482,
          0.40930328,  0.56072766],
        [-1.5080702 ,  0.54456604,  0.48591232, -0.42290282, -0.2829513 ,
         -0.17083132,  0.24723478],
        [ 0.32751667, -0.06909415,  0.5061601 ,  1.0977703 ,  0.30815402,
         -0.6388907 , -0.5418427 ],
        [-0.3520297 , -0.2602532 , -0.64145595, -0.49479467, -0.28470394,
          0.3443492 ,  0.4692584 ],
```

```
       [-0.37946287, -0.9158573 ,  0.30230346,  0.74018407,  0.57628316,
         0.24941704, -0.8486707 ],
       [-0.02487067, -0.3225647 ,  0.39394197,  0.06952756, -0.5112442 ,
         0.18681513, -0.6378209 ],
       [ 0.00904966,  0.14027253,  0.51242244, -0.5468718 ,  0.58860093,
        -1.1286767 , -0.29736024],
       [-0.685769  , -0.08927196,  0.27319017, -0.26816353, -0.6914297 ,
        -0.4507087 ,  0.13184133]], dtype=float32)
```